Mini Project Report On

AI-Based Resume Ranking and Candidate Shortlisting System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Pranav C Ajith (U2303182)

Pranav P Narayanan (U2303184)

Sidharth Revi (U2303207)

Satyam Sharma (U2303218)

Under the guidance of

Ms.Sangeetha Jamal

Department of Computer Science & Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

This is to certify that the mini project report entitled ***AI-Based Resume Ranking and Candidate Shortlisting System*** *is a bonafide record of the work done by* ***Pranav C Ajith (U2303182)***, ***Pranav P Narayanan (U2303184)***, ***Sidharth Revi (U2303207)***, ***Satyam Sharma (U2303218)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Ms.Sangeetha Jamal
Assistant Professor
Dept. of CSE
RSET

Ms.Mehbooba P Shareef
Assistant Professor
Dept. of CSE
RSET

Dr. Preetha K G
Head of the Department
Dept. of CSE
RSET

ACKNOWLEDGEMENT

We wish to express our sincere gratitude towards Rev.Fr.Dr.Jaison Paul Mulerikkal CMI, Principal of RSET, and Dr. Preetha K.G., Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our mini project, "AI-Based Resume Ranking and Candidate Shortlisting System".

We are highly indebted to our project coordinators, **Ms.Mehbooba P Shareef**, Assistant Professor, Department of Computer Science and Engineering for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms.Sangeetha Jamal**,Assistant Professor, Department of Computer Science and Engineering for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, We would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Pranav C Ajith

Pranav P Narayanan

Sidharth Revi

Satyam Sharma

Abstract

The AI-Based Resume Ranking and Candidate Shortlisting System is designed to automate and improve the recruitment process by intelligently analyzing and ranking job applicants based on their suitability for specific job roles. Traditional recruitment methods require recruiters to manually review large numbers of resumes, which is time-consuming, inefficient, and often affected by human bias. This project proposes an intelligent platform that uses Artificial Intelligence, Natural Language Processing (NLP), Large Language Models (LLMs), and Retrieval Augmented Generation (RAG) to analyze resumes and compare them with job descriptions.

The system allows recruiters to create job postings and candidates to upload their resumes through an interactive web platform. Once a resume is submitted, the system performs resume parsing and skill extraction to identify important information such as technical skills, experience, and keywords. A ranking algorithm then calculates similarity scores between the extracted resume features and the job requirements. Based on these scores, candidates are automatically ranked according to their relevance to the job role. In addition, the system uses LLMs integrated with RAG to generate explanations that highlight matched skills and missing qualifications, helping recruiters understand the reasoning behind each candidate's ranking.

Compared to traditional resume screening systems, the proposed system provides several advantages. It significantly reduces manual effort and screening time, improves accuracy in candidate-job matching, and enhances transparency in recruitment decisions through AI-generated explanations. The integration of modern AI techniques also enables the system to handle large volumes of applications efficiently while maintaining consistent evaluation criteria.

Overall, the project demonstrates how AI-driven automation can transform recruitment by enabling faster, fairer, and more data-driven hiring decisions.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **v**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 1

1.3 Scope and Motivation 2

1.4 Objectives 2

1.5 Challenges 3

1.6 Assumptions 3

1.7 Societal / Industrial Relevance 4

1.8 Organization of the Report 4

2 Software Requirements Specification **5**

2.1 Introduction 5

2.2 Overall Description 5

2.3 External Interface Requirements 7

2.4 Other Nonfunctional Requirements 8

3 System Architecture and Design **10**

3.1 System Overview 10

3.2 Architectural Design 11

3.3 Proposed Methodology/Algorithms 13

3.4 User Interface Design 14

3.5 Database Design 16

3.5.1 Users Table 16

3.5.2 Jobs Table 16
3.5.3 Applications Table 16
3.6 Description of Implementation Strategies 17
3.6.1 Resume Parsing 17
3.6.2 Ranking Engine 18
3.6.3 Backend Implementation 18
3.7 Module Division 19
3.8 Work Schedule - Gantt Chart 25

4 System Implementation, Overview and Testing 26
4.1 System Implementation 26
4.2 System Testing 26
4.2.1 Functional Testing 27
4.2.2 Resume Parsing Testing 27
4.2.3 Ranking Engine Testing 28
4.2.4 AI Feedback Testing 28
4.2.5 Integration Testing 29
4.3 Results and Discussion 29
4.3.1 Resume Parsing Results 29
4.3.2 Candidate Ranking Results 29
4.3.3 AI Explanation and Feedback 30
4.3.4 User Interface Results 31

5 Conclusion 33
5.1 Conclusion 33
5.2 Future Work 33

Appendix A: Presentation 36

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes 55

Vision, Mission, POs, PSOs and COs ii

Appendix C: CO-PO-PSO Mapping vi

List of Figures

3.1 Architecture Diagram of Resume Ranker 11
3.2 Job Opening 14
3.3 Resume Upload 14
3.4 Resume Evaluation In Pending Stage 15
3.5 Ranking 15
3.6 Result and LLM Feedback 15
3.7 Work Division 25

4.1 Candidate Dashboard 32
4.2 Recruiter Dashboard 32

Chapter 1

Introduction

1.1 Background

In modern recruitment processes, organizations receive a large number of job applications for every open position. Recruiters are often required to manually review hundreds or even thousands of resumes to identify suitable candidates. This manual screening process is time-consuming, inefficient, and prone to human bias or oversight. As the number of applicants continues to increase due to the growth of online job platforms, traditional resume evaluation methods are becoming less practical.

Recent advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP) have made it possible to automate several stages of the recruitment process. Resume parsing systems can extract important information such as skills, qualifications, and experience from resumes and compare them with job requirements. AI-driven solutions can help recruiters quickly identify the most relevant candidates, thereby reducing the workload and improving the efficiency of the hiring process.

The proposed AI-Based Resume Ranking and Candidate Shortlisting System aims to simplify the recruitment process by automatically analyzing resumes and ranking candidates according to their relevance to job descriptions. The system uses NLP techniques for resume processing and integrates modern AI models such as Large Language Models (LLMs) and Retrieval Augmented Generation (RAG) to generate explanations for candidate rankings. This improves transparency and assists recruiters in making faster and more informed hiring decisions.

1.2 Problem Definition

The aim of this project is to develop an AI-powered system that automatically analyzes resumes, compares them with job descriptions, and ranks candidates based on their suit-

ability for the job role.

The system is designed to reduce manual effort in resume screening and improve the efficiency and accuracy of candidate shortlisting in the recruitment process.

1.3 Scope and Motivation

The scope of this project is to design and implement a web-based platform that automates resume analysis and candidate ranking. The system allows recruiters to create job openings and candidates to upload resumes through an online interface. It extracts important information from resumes such as skills, qualifications, and experience using Natural Language Processing techniques. The system then compares this information with job requirements and calculates a matching score to rank candidates accordingly. Additionally, the system generates AI-based explanations that describe why a candidate is suitable or unsuitable for a specific job role.

The primary motivation for this project arises from the challenges faced by recruiters when dealing with large volumes of job applications. Manual resume screening consumes significant time and effort and may lead to inconsistent evaluation of candidates. By introducing AI-based automation, organizations can significantly improve the speed and accuracy of candidate selection. Furthermore, intelligent systems can help reduce bias and ensure fair evaluation based on relevant skills and qualifications. This project aims to demonstrate how modern AI technologies can enhance recruitment efficiency and support better hiring decisions..

1.4 Objectives

The project aims to develop an automated system for resume screening and candidate evaluation, enabling recruiters to efficiently handle large volumes of applications with minimal manual effort. By automating the initial stages of recruitment, the system reduces dependency on time-consuming manual processes and enhances overall productivity.

A key objective of the system is to extract relevant information such as skills, educational qualifications, and professional experience from resumes using advanced Natural Language Processing (NLP) techniques. This ensures that important candidate details

are accurately identified and structured for further analysis, regardless of variations in resume formats.

The system is also designed to incorporate a robust ranking algorithm that compares the extracted resume data with specific job requirements. Based on this comparison, it assigns matching scores to candidates, helping recruiters quickly identify the most suitable applicants for a given role. This data-driven approach improves the quality and consistency of candidate selection.

In addition, a web-based interface will be developed to provide recruiters with an intuitive platform for posting job openings, managing applications, and reviewing candidate profiles. This interface aims to streamline the recruitment workflow and offer better organization and accessibility of information.

Another important objective is to generate AI-based explanations for candidate rankings using Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) techniques. This feature enhances transparency by providing clear justifications for each candidate's ranking, thereby building trust in the system's decisions.

1.5 Challenges

One of the major challenges in this project is handling the variation in resume formats, which makes information extraction difficult. Another challenge is ensuring accurate skill extraction and matching, as candidates may describe skills in different ways. Integrating multiple technologies such as AI models, backend APIs, databases, and frontend frameworks also increases system complexity.

1.6 Assumptions

Resumes are uploaded in supported formats such as PDF for processing. Job descriptions provided by recruiters clearly specify required skills and qualifications. Candidates provide accurate and truthful information in their resumes. The system assumes stable internet connectivity for accessing the platform.

1.7 Societal / Industrial Relevance

The proposed system has significant relevance in both industry and society. In the corporate sector, companies frequently receive large numbers of applications for job openings, making manual resume screening inefficient. This system can help organizations automate the recruitment process and quickly identify the most suitable candidates. It also improves transparency by providing explanations for candidate rankings.

From a societal perspective, AI-driven recruitment platforms can promote fair and unbiased hiring practices by evaluating candidates based on their skills and qualifications rather than subjective human judgment. This can lead to more efficient hiring processes and improved opportunities for job seekers.

1.8 Organization of the Report

This report is organized into five main chapters, each focusing on different aspects of the proposed system.

Chapter 1: Introduction This chapter presents the background of the project, problem definition, scope and motivation, objectives, challenges, assumptions, and the societal or industrial relevance of the system.

Chapter 2: Software Requirements Specification This chapter describes the functional and non-functional requirements of the system. It includes the overall system description, external interface requirements, system features, and constraints.

Chapter 3: System Architecture and Design This chapter explains the overall system architecture, design components, and methodology used in the project. It covers the architectural design, dataset details, algorithms, user interface design, database structure, and module descriptions.

Chapter 4: System Implementation, Testing, and Results This chapter discusses the implementation details of the system, including the technologies used and workflow. It also presents various testing methods such as functional testing, integration testing, and module testing, along with the results and performance analysis.

Chapter 5: Conclusion and Future Work This chapter summarizes the outcomes of the project and highlights its contributions. It also suggests possible future enhancements and improvements to the system.

Chapter 2

Software Requirements Specification

2.1 Introduction

The rapid growth of job applications in modern recruitment processes has made manual resume screening inefficient and time-consuming. Recruiters often need to evaluate hundreds of resumes to identify suitable candidates for a job role. This process can lead to delays, human bias, and inconsistent candidate evaluation.

The AI Resume Ranking System is designed to automate the resume screening process by analyzing candidate resumes and comparing them with job requirements. The system uses Natural Language Processing techniques to extract important information such as skills and experience from resumes. Based on this information, the system calculates a ranking score for each candidate according to how well their profile matches the job description.

The system provides two main user roles: Candidate and Recruiter. Candidates can upload resumes and apply for jobs, while recruiters can create job postings and view ranked candidate lists. Additionally, the system provides AI-based feedback explaining the ranking results and suggesting skill improvements.

The goal of this system is to improve recruitment efficiency, reduce manual workload, and provide transparent evaluation of candidates.

2.2 Overall Description

The AI Resume Ranking System is a web-based application consisting of a frontend user interface, backend server, and database system.

The system architecture includes the following components:

a. Frontend Interface

The Frontend Interface, developed using React, serves as the user-facing layer of the system. It provides an interactive and responsive environment for both candidates and recruiters. Candidates can upload their resumes, view job listings, and track their application status, while recruiters can post job openings, review applications, and access ranked candidate lists. The interface is designed to be intuitive and user-friendly, ensuring seamless navigation and accessibility across different devices.

b. Backend Server

The Backend Server, implemented using FastAPI, acts as the core processing unit of the system. It handles all API requests from the frontend and manages the overall system logic. This includes processing resume uploads, coordinating with the parsing and ranking modules, and retrieving data from the database. FastAPI ensures high performance and quick response times, making the system efficient and scalable for handling multiple users simultaneously.

c. Resume Parser

The Resume Parser is responsible for analyzing uploaded resumes and extracting relevant information such as skills, educational qualifications, work experience, and other key attributes. It uses Natural Language Processing (NLP) techniques to convert unstructured resume data into structured formats that can be easily processed by other components of the system.

d. Ranking Engine

The Ranking Engine evaluates candidates by comparing the extracted resume data with the requirements specified in job postings. It uses a scoring algorithm to measure how well a candidate's skills and experience match the job criteria. Factors such as skill relevance, experience level, and keyword matching are considered to generate a ranking score for each candidate.

e. AI Explanation Module

The AI Explanation Module enhances transparency by generating meaningful feedback for each candidate's ranking. Instead of just providing a score, it explains why a candidate was ranked in a particular position, highlighting strengths, missing skills, and areas for improvement. This helps recruiters make informed decisions and also provides constructive insights to candidates.

f. Database

The Database is used to store all system-related data, including user accounts, resumes, job postings, and application records. It ensures data consistency, security, and efficient retrieval. The structured storage allows seamless interaction between different system components and supports long-term data management.

The system automatically processes uploaded resumes, evaluates candidates based on job requirements, and provides recruiters with ranked candidate lists.

2.3 External Interface Requirements

External interface requirements describe how the system interacts with hardware devices, software components, and external services. This section explains the hardware requirements, software environment, APIs used, and communication mechanisms used in the system.

Hardware Interface

The system runs on standard computer systems and does not require any specialized hardware. The minimum hardware requirements for running the application efficiently are listed below:

Processor: Intel i3 or higher

RAM: 4 GB minimum

Storage: 10 GB available space

These specifications ensure that the system can process resumes, perform ranking operations, and run the backend services smoothly.

Software Interface

The system is developed using modern web technologies and requires the following software environment:

Operating System: Windows / Linux / macOS

Frontend: React.js (used for building the interactive user interface)

Backend: FastAPI (Python framework used to handle API requests and application logic)

Database: SQLite (used to store users, job postings, and application data)

These software components work together to provide a scalable and efficient recruit-

ment platform.

APIs Used

Several libraries and APIs are used to process and analyze resumes:

PyPDF is used to extract text content directly from PDF resumes in a structured and readable format.

OCR (Optical Character Recognition) is used to convert text from image-based resumes (scanned documents) into machine-readable text.

Regex (Regular Expressions) is used to identify and extract specific patterns such as email addresses, phone numbers, and technical skills from the parsed text.

These tools enable the system to automatically analyze resumes and extract relevant candidate information.

Communication Interface

Communication between the frontend and backend components occurs through RESTful API requests. The frontend sends HTTP requests to the backend, and the backend processes the data and returns the results in JSON format.

Example API endpoints used in the system include:

POST /resumes/upload

GET /ranking/opening/{job_id}

POST /recruiter/update_status/{id}

These APIs allow users to upload resumes, retrieve ranked candidate lists for specific job openings, and update the application status of candidates.

2.4 Other Nonfunctional Requirements

Performance Requirements

The system should process and rank resumes within a few seconds after upload.

Security Requirements

The system is designed for fast performance, allowing resumes to be processed and ranked within a few seconds after upload, ensuring quick results for recruiters.

Reliability

The system ensures reliability by securely storing resumes and application data with proper data management, reducing the risk of data loss.

Usability

In terms of usability, the interface is simple and intuitive, making it easy for users to navigate and perform tasks without difficulty.

Scalability

The architecture also supports scalability, allowing the system to handle an increasing number of users and job postings as it grows.

Chapter 3

System Architecture and Design

3.1 System Overview

The AI Resume Ranking System is designed to automate the resume screening process using artificial intelligence techniques. The system architecture consists of a frontend interface, backend server, resume processing module, ranking engine, AI explanation system, and database.

The workflow of the system is as follows:

1. A candidate uploads a resume through the frontend interface.
2. The backend server receives the resume and passes it to the resume parser.
3. The resume parser extracts relevant information such as skills and experience.
4. The ranking engine compares the candidate's profile with job requirements.
5. A score is calculated based on skill matching and experience.
6. The AI explanation module generates feedback for the candidate.
7. The system stores the results in the database and displays them on the dashboard.

3.2 Architectural Design

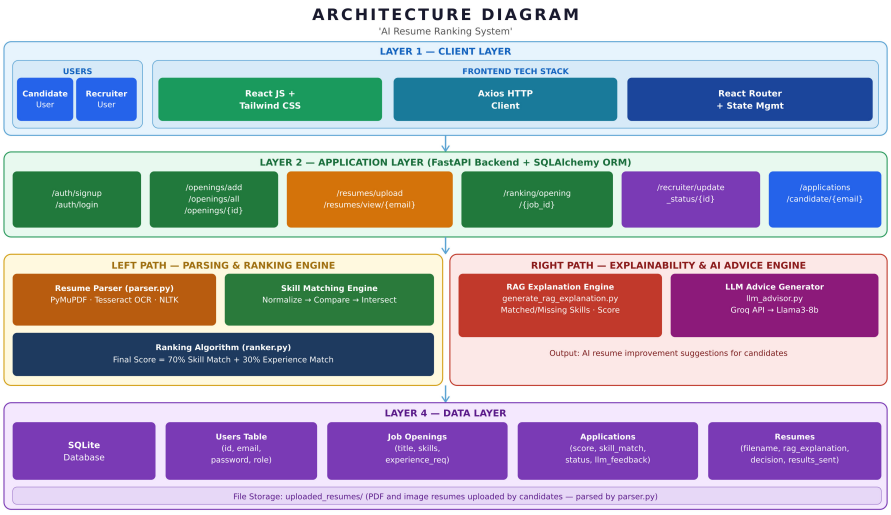


Figure 3.1: Architecture Diagram of Resume Ranker

LAYER 1 — Client Layer (What Users See & Interact With)

This is the front-end, which is the website that users access through their browser.

There are two types of users in the system. The Candidate User is a job seeker who creates an account, uploads their resume (PDF or image), and views AI-generated feedback to improve it. The Recruiter User is an HR representative who posts job openings, views ranked candidate lists, and updates application statuses such as shortlisted or rejected.

The frontend is built using React JS and Tailwind CSS for designing and styling the interface. Axios is used to send HTTP requests from the browser to the backend server, such as uploading resumes or fetching rankings. React Router and state management are used to handle page navigation and maintain user session information.

LAYER 2 — Application Layer (The Backend Brain / API)

This is the server-side backend developed using FastAPI along with SQLAlchemy ORM for database interaction. It exposes several API endpoints to handle different functionalities.

The authentication endpoints (**/auth/signup** and **/auth/login**) are used for secure user registration and login. The job openings endpoints (**/openings/add**, **/openings/all**, **/openings/{id}**) allow recruiters to create, view, and manage job postings.

The resume endpoints (**/resumes/upload** and **/resumes/view/{email}**) handle resume uploads by candidates and viewing by recruiters.

The ranking endpoint (**/ranking/opening/{job_id}**) triggers the AI engine to rank candidates for a specific job. The recruiter endpoint (**/recruiter/update_status/{id}**) allows updating candidate status. The applications endpoint (**/applications/candidate/{email}**) enables candidates to view all their job applications and results.

LAYER 3 — AI Processing Layer (Where the Intelligence Lives)

This layer consists of two parallel engines that operate after a resume is uploaded.

Parsing and Ranking Engine

This part is responsible for reading and evaluating the resume. The Resume Parser (parser.py) processes the uploaded file by extracting text from PDFs using PyMuPDF and from image-based resumes using Tesseract OCR. The extracted text is cleaned and processed using NLP techniques such as NLTK.

The Skill Matching Engine compares the candidate's skills with the job requirements. It normalizes terms (for example, converting "JS" to "JavaScript") and identifies matching skills.

The Ranking Algorithm (ranker.py) computes a final score. Seventy percent of the score is based on skill matching, and thirty percent is based on how well the candidate's experience aligns with the job requirements.

Explainability and AI Advice Engine

This part explains the ranking results and provides feedback. The RAG Explanation Engine (generate_rag_explanation.py) generates a clear explanation showing matched and missing skills and what the score represents.

The LLM Advice Generator (llm_advisor.py) sends this explanation to the Llama3-8b model via the Groq API. The model then generates personalized suggestions, such as adding missing skills or including relevant projects.

The final output provides candidates with clear and actionable advice to improve their resumes.

LAYER 4 — Data Layer (Where Everything is Stored)

This layer handles data storage using SQLite, which is a lightweight database.

The Users table stores user details such as ID, email, hashed password, and role (candidate or recruiter). The Job Openings table stores job titles, required skills, and experience levels. The Applications table records skill match scores, AI feedback, and application status (pending, accepted, or rejected).

The Resumes table stores file names, AI-generated explanations, hiring decisions, and result status. Additionally, the actual resume files (PDFs and images) are stored in a directory such as `uploaded_resumes/`, which is accessed by the parser.

Overall Workflow (End-to-End Flow)

The system operates in a sequence of steps. A recruiter logs in and posts a job opening with required skills and experience. A candidate logs in and uploads their resume. The system parses the resume and evaluates it using the ranking engine.

The AI modules then generate an explanation and suggestions for improvement. All data is stored in the SQLite database. The recruiter can view ranked candidates and update their application status. The candidate can later log in to view their score, matched and missing skills, and AI-generated advice for improvement.

3.3 Proposed Methodology/Algorithms

The proposed methodology for the system involves the following steps:

Algorithm: Resume Ranking Algorithm

Input: Resume, Job Skills Output: Ranking Score

Step 1: Upload resume

Step 2: Extract resume text

Step 3: Identify skills from resume

Step 4: Extract experience duration

Step 5: Compare resume skills with job skills

Step 6: Calculate skill match percentage

Step 7: Compute ranking score

Step 8: Generate AI explanation

Step 9: Display result to user

Score Calculation:

Score =70 Skill Match + 30 Experience Match

3.4 User Interface Design

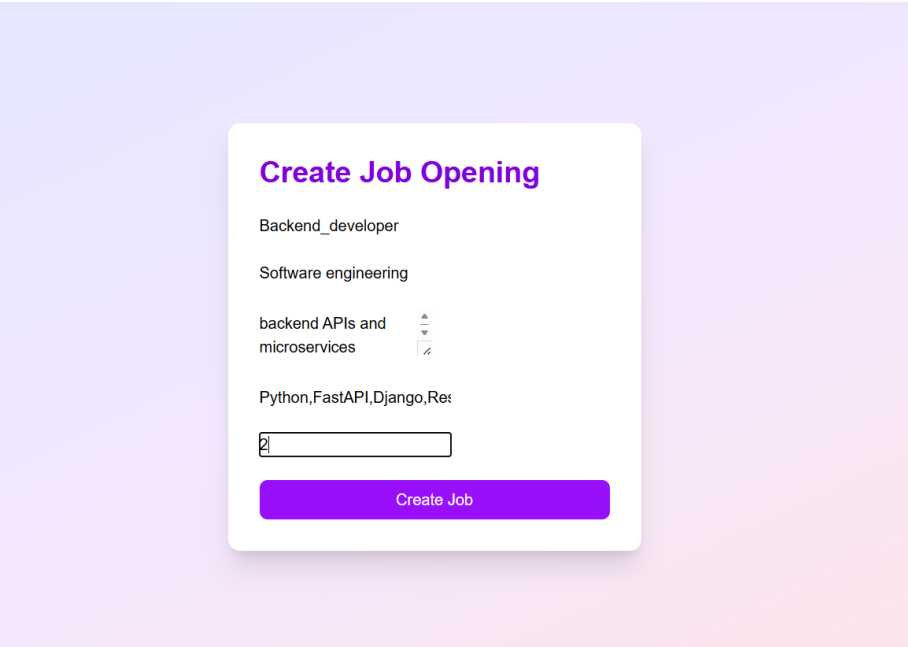


Figure 3.2: Job Opening

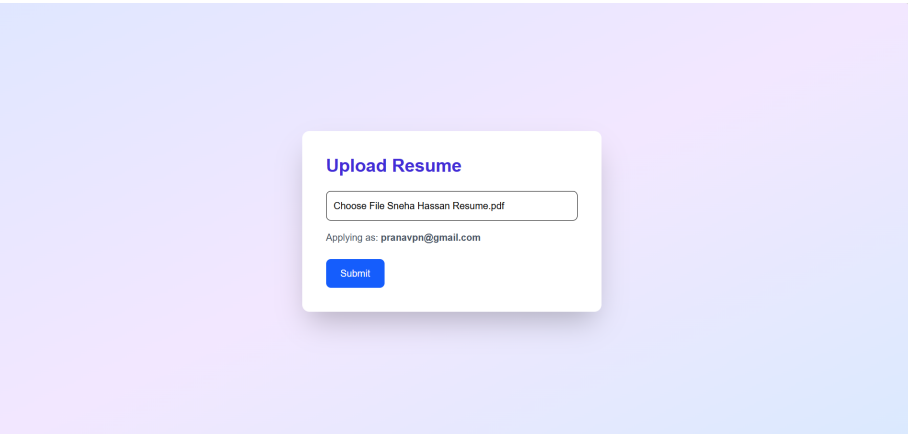


Figure 3.3: Resume Upload

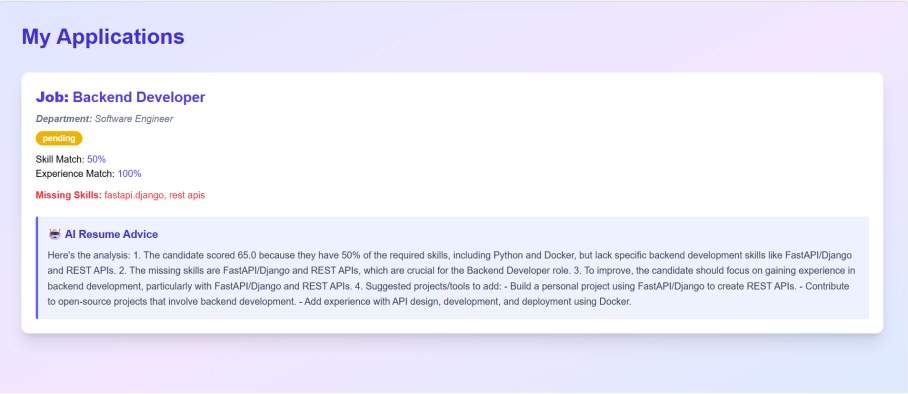


Figure 3.4: Resume Evaluation In Pending Stage

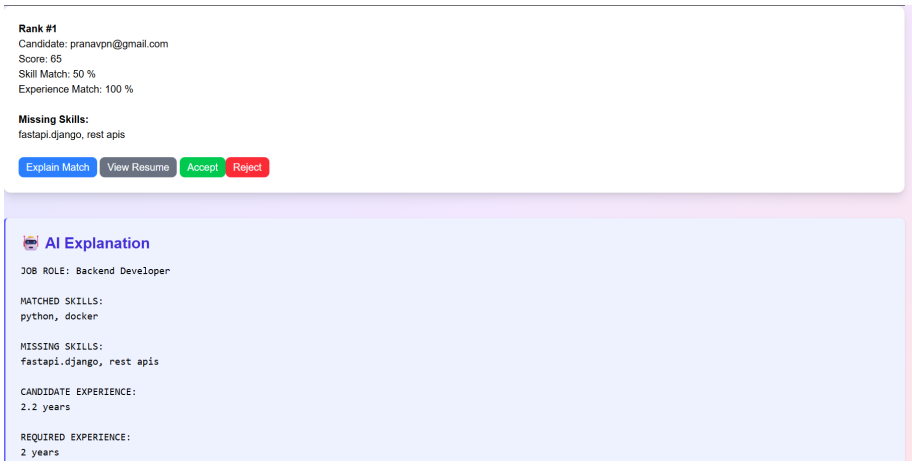


Figure 3.5: Ranking

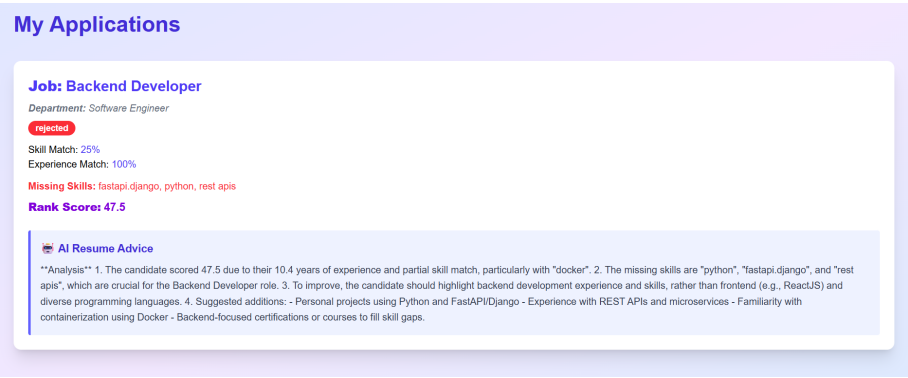


Figure 3.6: Result and LLM Feedback

3.5 Database Design

The system uses **SQLite** as the database for storing application data. SQLite was chosen because it is lightweight, easy to integrate with Python-based applications, and does not require a separate server configuration.

The database consists of three main tables: **Users**, **Jobs**, and **Applications**. These tables store information related to users, job postings, and candidate applications.

3.5.1 Users Table

This table stores the account information of both candidates and recruiters.

User ID	Name	Email	Password	Role

Table 3.1: Users Table Schema

3.5.2 Jobs Table

This table stores job postings created by recruiters, including job details and required skills.

Job ID	Job Title	Job Description	Required Skills	Recruiter ID

Table 3.2: Jobs Table Schema

3.5.3 Applications Table

This table stores the applications submitted by candidates for different job postings.

Application ID	Candidate Email	Job ID	Resume Path	Ranking Score
Application Status				

Table 3.3: Applications Table Schema

3.6 Description of Implementation Strategies

The system is implemented using modern web development technologies to automate the process of resume analysis and candidate ranking. The architecture mainly consists of three modules: Resume Parsing, Ranking Engine, and Backend Implementation.

3.6.1 Resume Parsing

Resume parsing is the process of extracting meaningful information from resumes such as skills, work experience, education, and contact details. The system accepts resumes in formats such as PDF or image files and converts them into structured text data that can be analyzed programmatically.

Libraries Used:

- **PyPDF** – Used to read and extract text from PDF documents.
- **OCR (Optical Character Recognition)** – Used when resumes are uploaded as scanned images. OCR converts the image content into machine-readable text.
- **Regex (Regular Expressions)** – Used to identify patterns in the extracted text and extract specific information such as phone numbers, email addresses, skills, and experience details.

The parsing module processes the extracted text and identifies important attributes such as candidate skills and work experience.

Example Code Snippet:

```
text = extract_text_from_resume(file)


skills = extract_skills(text)
experience = extract_experience(text)
```

In this step, the resume text is first extracted and then processed using predefined rules to identify relevant skills and experience information.

3.6.2 Ranking Engine

The ranking engine evaluates and ranks candidates based on how well their resumes match the job requirements. It compares the extracted candidate skills and experience with the required job description.

A scoring system is used to compute the final ranking score. Skills are given higher priority because they directly represent the candidate's technical capabilities, while experience contributes to the practical knowledge of the candidate.

The scoring formula used is:

$$score = (skill_match \times 0.7) + (experience_score \times 0.3)$$

Where:

- **skill_match** represents the percentage of required skills found in the resume.
- **experience_score** represents the relevance and duration of the candidate's work experience.

This weighted scoring system ensures that candidates with strong skill alignment and relevant experience receive higher rankings.

3.6.3 Backend Implementation

The backend of the system is developed using **FastAPI**, a modern Python web framework that enables fast and efficient API development. FastAPI is chosen because of its high performance, asynchronous capabilities, and automatic API documentation.

The backend handles tasks such as:

- Resume upload and processing
- Resume parsing
- Skill extraction
- Candidate ranking
- Returning ranked candidate results

Example API Endpoint:

```
from fastapi import FastAPI, UploadFile

app = FastAPI()

@app.post("/resumes/upload")
def upload_resume(file: UploadFile):
    text = process_resume(file)
    return {"message": "Resume processed successfully"}
```

In this endpoint, the user uploads a resume file, which is then processed by the system. The extracted information is stored and used for ranking candidates against job descriptions.

Overall, this architecture enables efficient resume analysis and automated candidate evaluation, significantly reducing manual screening effort in recruitment processes.

3.7 Module Division

The AI Resume Ranking System is divided into several functional modules to ensure modular development, easier maintenance, and efficient system operation. Each module performs a specific task within the system architecture and interacts with other modules to complete the resume analysis and ranking process. The following modules constitute the overall system.

1. User Authentication Module

The User Authentication Module manages the registration and login functionalities of the system. It ensures secure access to the application by validating user credentials and maintaining separate roles for candidates and recruiters.

The module allows new users to create accounts and existing users to log into the system using their registered email and password. Based on the user role, the system redirects users to the appropriate dashboard. This module ensures that only authorized users can access the platform and prevents unauthorized access to sensitive data.

The authentication process involves password hashing and secure credential storage in the database. The module interacts with the **Users table** in the database for verifying login details and storing new user accounts.

Key Functions

- User registration
- User login and authentication
- Role-based access control
- Session management

2. Resume Upload and Management Module

The Resume Upload Module allows candidates to upload their resumes in supported formats such as PDF or image files. Once the resume is uploaded, the system stores the file in the server storage and records the relevant details in the database.

This module handles file management operations including storing the uploaded resumes in designated folders and linking them with candidate profiles and job applications. It also validates file formats and ensures proper file handling to prevent system errors.

The module interacts with the backend API to send resume files for processing and parsing.

Key Functions

- Resume file upload
- File storage management
- Resume metadata storage
- Linking resumes with job applications

3. Resume Parsing Module

The Resume Parsing Module is responsible for extracting structured information from uploaded resumes. Since resumes are usually unstructured documents, the system uses

Natural Language Processing techniques and text extraction methods to identify important information.

This module processes the resume file to extract the text content and then identifies relevant data such as candidate skills, work experience, and other professional information. It uses regular expressions and keyword matching techniques to detect skills and employment periods from the resume text.

The extracted information is then converted into a structured format that can be used by the ranking engine for further analysis.

Key Functions

- Text extraction from resume files
- Skill identification and extraction
- Experience calculation
- Data cleaning and normalization

Technologies Used

- Python text processing libraries
- OCR tools for image-based resumes
- Regular expressions for pattern matching

4. Candidate Ranking Engine Module

The Ranking Engine is one of the core components of the system. It evaluates candidates by comparing the extracted resume information with the job requirements specified by recruiters.

The ranking engine calculates a score based on two main factors: skill matching and work experience. The system determines how many required job skills appear in the candidate's resume and calculates a skill matching percentage. It also evaluates the candidate's work experience relative to the job's experience requirements.

The final ranking score is computed using a weighted scoring formula.

Example Scoring Formula

$$FinalScore = (0.7 \times SkillMatchScore) + (0.3 \times ExperienceScore)$$

Candidates are then ranked according to their scores so that recruiters can easily identify the most suitable applicants.

Key Functions

- Skill matching between resumes and job descriptions
- Experience comparison
- Candidate scoring and ranking
- Generation of matched and missing skill lists

5. AI Explanation and Feedback Module

The AI Explanation Module enhances the system by providing detailed feedback about the ranking results. Instead of displaying only numerical scores, this module generates human-readable explanations that help candidates understand why their resumes received a particular score.

The module analyzes the ranking results and highlights matched skills and missing skills. It also provides suggestions for improving the resume by recommending additional skills or qualifications that could improve the candidate's match score.

This module uses Retrieval Augmented Generation (RAG) combined with a Large Language Model (LLM) to generate contextual explanations based on the ranking results.

Key Functions

- Generating explanation for ranking results
- Highlighting matched skills
- Identifying missing skills
- Providing improvement suggestions for candidates

6. Candidate Dashboard Module

The Candidate Dashboard provides an interactive interface for job seekers to interact with the system. Through this dashboard, candidates can browse available job openings, upload resumes, and track their application status.

The dashboard displays the results of the resume ranking process, including the candidate's match score, matched skills, missing skills, and AI-generated feedback.

Key Functions

- Viewing job listings
- Applying for jobs
- Uploading resumes
- Viewing ranking results and AI feedback
- Tracking application status

7. Recruiter Dashboard Module

The Recruiter Dashboard provides functionality for recruiters to manage job postings and review candidate applications. Recruiters can create new job openings by specifying job descriptions, required skills, and experience requirements.

The dashboard displays a ranked list of candidates who have applied for a particular job.

Key Functions

- Job posting creation
- Viewing candidate applications
- Reviewing ranked candidate lists
- Accepting or rejecting applications

8. Database Management Module

The Database Management Module is responsible for storing and managing all system data. The system uses a relational database to store user information, job postings, resumes, and application data.

Major Database Tables

- Users table
- Job Openings table
- Applications table
- Resume records table

The database interacts with the backend server through structured queries and ensures that data is stored securely and consistently.

Work Allocation Among Team Members

The project work was divided among the team members to ensure efficient development and timely completion of the system.

Team Member	Module Responsibility
Pranav P	Frontend development and Candidate Dashboard implementation
Sidharth	Resume parsing and skill extraction module
Pranav C	Ranking engine development and AI explanation module
Satyam	Backend API development and database management

Each member contributed to system integration, testing, debugging, and final documentation of the project.

3.8 Work Schedule - Gantt Chart

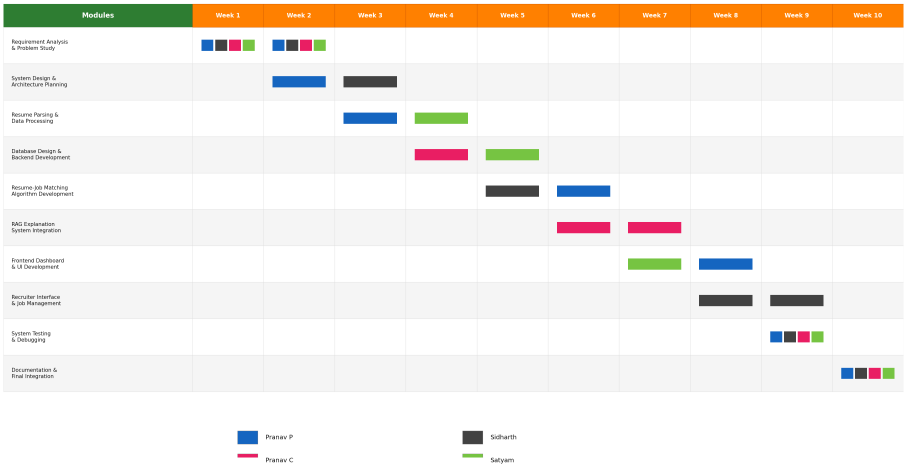


Figure 3.7: Work Division

Chapter 4

System Implementation, Overview and Testing

4.1 System Implementation

The proposed AI Resume Ranking System was successfully implemented as a web-based application that automates the process of resume screening and candidate evaluation. The system consists of a React-based frontend interface, a FastAPI backend server, and a SQLite database for storing user information, job openings, and candidate applications.

The system allows candidates to upload resumes and apply for job openings while recruiters can view candidate applications and ranked results. The backend processes the uploaded resumes using a resume parser that extracts relevant information such as skills and work experience. The ranking engine then evaluates the candidate by comparing extracted skills with the required skills for the job.

The overall workflow of the system is as follows:

1. The candidate logs into the system and browses available job openings.
2. The candidate uploads a resume and applies for a job.
3. The system parses the resume and extracts skills and experience.
4. The ranking engine compares the extracted data with job requirements.
5. A ranking score is generated based on skill matching and experience.
6. AI-generated feedback explaining the ranking result is produced.
7. The results are displayed on the candidate dashboard and stored in the database.
8. Recruiters can view ranked candidate lists and make hiring decisions.

4.2 System Testing

System testing was performed to evaluate the functionality and reliability of the developed AI Resume Ranking System. Various test cases were designed to verify whether each module of the system operates correctly and produces the expected results.

The system was tested using multiple resumes and job descriptions to validate the resume parsing process, ranking engine calculations, AI feedback generation, and overall application workflow.

The following testing methods were applied during the development process.

4.2.1 Functional Testing

Functional testing ensures that each feature of the system performs according to its intended purpose. The following functionalities were tested successfully:

User registration and login for both candidates and recruiters.

Job posting functionality for recruiters.

Resume upload functionality for candidates.

Resume parsing and skill extraction from uploaded files.

Candidate ranking based on skill match and experience.

Display of ranking results in candidate dashboard.

Viewing ranked applicants in recruiter dashboard.

Accepting or rejecting candidates by recruiters.

Each feature was tested using multiple inputs to ensure correct functionality and reliable output.

4.2.2 Resume Parsing Testing

The resume parser was tested with different types of resumes including text-based PDFs and scanned resumes. The system successfully extracted important information such as skills and experience.

For example:

Input Resume Skills:

Python, Java, SQL, Machine Learning

Extracted Skills:

Python, Java, SQL, Machine Learning

Extracted Experience:

4.5 years

This confirmed that the parser module can correctly identify relevant information from resumes.

4.2.3 Ranking Engine Testing

The ranking engine was tested by applying resumes to different job openings with varying skill requirements. The objective of the test was to verify that the system correctly compares candidate skills with the required job skills and calculates the ranking score accordingly.

Example Test Case

Parameter	**Value**
Job Required Skills	Python, Machine Learning, Docker
Candidate Resume Skills	Python, Java
Matched Skills	Python
Missing Skills	Machine Learning, Docker
Skill Match Score	33%

Table 4.1: Ranking Engine Test Case

The final ranking score was calculated using the scoring formula implemented in the system. The result confirmed that the ranking engine correctly identifies matched skills, determines missing skills, and calculates the appropriate ranking score based on the defined weighting criteria.

4.2.4 AI Feedback Testing

The AI explanation module was tested to ensure that it generates meaningful explanations for ranking results. The module successfully generated explanations describing matched skills, missing skills, and suggestions for improvement.

Example output:

Your resume matches the Python skill required for this role. However, the job description also requires Machine Learning and Docker. Learning these technologies can significantly improve your ranking score.

This feature improves transparency in the recruitment process by explaining how the ranking score was generated.

4.2.5 Integration Testing

Integration testing was performed to ensure that all modules of the system work together correctly. The following workflow was tested:

1. Recruiter creates a job opening.
2. Candidate uploads a resume.
3. Resume is parsed by the backend.
4. Ranking score is calculated.
5. Results are stored in the database.
6. Candidate views results on dashboard.
7. Recruiter views ranked applicants and makes decisions.

The system successfully handled the entire workflow without errors.

4.3 Results and Discussion

4.3.1 Resume Parsing Results

The resume parser module was tested with multiple resumes in different formats such as PDF files. The parser successfully extracted relevant information including candidate skills and professional experience.

The skill extraction process identifies keywords from the resume and matches them with a predefined skill ontology. The experience extraction module calculates the total duration of professional experience by identifying date ranges within the resume.

Example extracted data from a sample resume:

Skills Detected: Python, Java, C++, AI Experience: 4.8 years

This structured data is then used by the ranking engine to evaluate candidate suitability for a specific job role.

4.3.2 Candidate Ranking Results

The ranking engine compares the extracted skills from the resume with the required skills specified in the job description. The ranking score is calculated using a weighted formula based on skill matching and work experience.

The scoring formula used in the system is:

$$FinalScore = (0.7 \times SkillMatchScore) + (0.3 \times ExperienceScore)$$

Where:

Skill Match Score represents the percentage of job skills found in the resume.

Experience Score represents how closely the candidate's experience matches the required experience for the job.

Example Evaluation

Parameter	**Value**
Job Required Skills	Python, Machine Learning, Java, Docker
Candidate Resume Skills	Python, Java
Matched Skills	Python, Java
Missing Skills	Machine Learning, Docker
Skill Match Score	$2/4 = 50\%$
Final Ranking Score	$FinalScore = (0.7 \times 50) + (0.3 \times ExperienceScore)$

Table 4.2: Example Candidate Ranking Evaluation

This ranking mechanism allows recruiters to quickly identify the most suitable candidates for a given job role by evaluating how closely their skills and experience match the job requirements.

4.3.3 AI Explanation and Feedback

The system also generates AI-based feedback explaining the ranking results. This explanation is generated using a Retrieval Augmented Generation (RAG) approach combined with a Large Language Model (LLM).

The system retrieves the following contextual information:

- Candidate skills
- Job requirements
- Matched skills

- Missing skills
- Ranking score

This information is passed to the language model, which generates a natural language explanation describing how well the candidate's resume matches the job requirements and suggesting areas for improvement.

Example AI Feedback

AI Generated Feedback
Your resume matches the Python and Java skills required for this role. However, the job description also requires Machine Learning and Docker. Improving your knowledge in these areas can significantly increase your match score for this position.

Table 4.3: Example AI Feedback Generated by the System

This feature helps candidates understand how to improve their resumes and increases the transparency of the ranking process by clearly explaining how the ranking score was calculated.

4.3.4 User Interface Results

The system provides two separate dashboards for candidates and recruiters.

Candidate Dashboard

The candidate interface allows users to:

Browse available job openings

Upload resumes

Apply for jobs

View ranking scores

Receive AI-generated feedback

Track application status

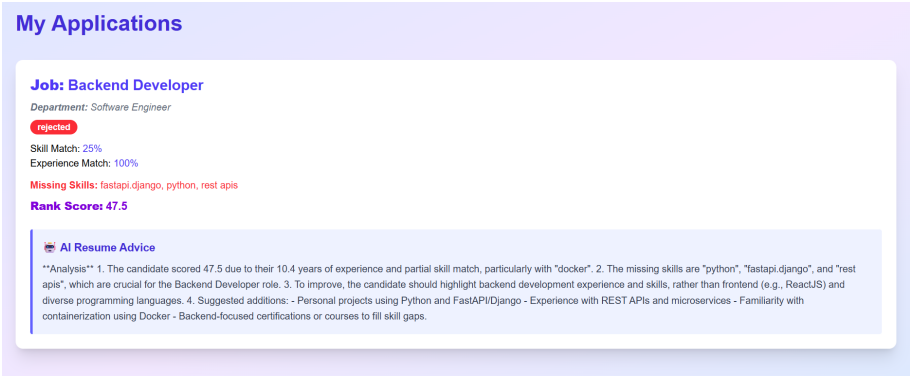


Figure 4.1: Candidate Dashboard

Recruiter Dashboard

The recruiter dashboard allows recruiters to:

Create job postings

View candidate applications

View ranked candidate lists

Review resumes

Accept or reject candidates

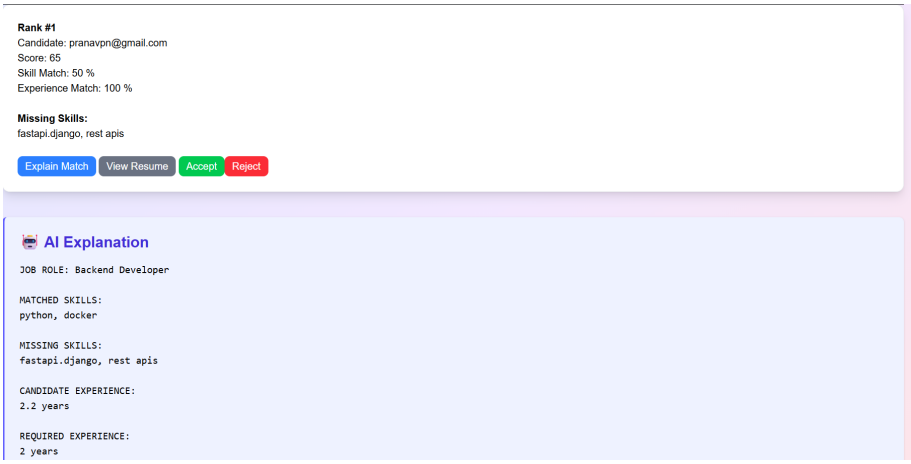


Figure 4.2: Recruiter Dashboard

Chapter 5

Conclusion

5.1 Conclusion

The AI Resume Ranking System developed in this project provides an efficient and automated solution for screening job applicants. The system reduces the manual effort required by recruiters to review resumes and enables faster identification of suitable candidates.

The proposed system successfully integrates resume parsing, skill extraction, candidate ranking, and AI-generated feedback into a unified platform. By analyzing resumes and comparing them with job requirements, the system generates ranking scores that help recruiters make better hiring decisions.

The implementation of Explainable AI through Retrieval Augmented Generation further enhances the system by providing meaningful explanations for candidate rankings. This improves transparency and helps candidates understand how their skills match job requirements.

The system also provides separate dashboards for candidates and recruiters, making the recruitment process more organized and user-friendly.

Overall, the developed system demonstrates how artificial intelligence techniques can be effectively applied to automate resume screening and improve the efficiency of recruitment processes.

5.2 Future Work

Although the current system performs resume ranking effectively, several improvements can be made in future versions of the project.

One possible enhancement is the integration of advanced Natural Language Processing techniques for deeper semantic analysis of resumes. This would allow the system to better

understand the context of skills and work experience.

Another improvement would be the use of machine learning models trained on large recruitment datasets to predict candidate suitability more accurately. This could further improve ranking accuracy and reduce dependency on rule-based skill matching.

Future versions of the system could also include automated interview scheduling and integration with online job portals to expand the recruitment workflow.

Additionally, support for multiple resume formats and multilingual resume parsing could be implemented to make the system more flexible and accessible.

Finally, deploying the system on a cloud platform would improve scalability and allow the system to support larger numbers of users and job applications.

Bibliography

[1] H. Garg and M. Dave, "Securing IoT Devices and SecurelyConnecting the Dots Using REST API and Middleware," 2019 4th International Conference on Internet of Things: Smart Innovation and Usages (IoT-SIU), Ghaziabad, India, 2019, pp. 1-6, doi: 10.1109/IoT-SIU.2019.8777334.

[2] M. Ebrahimi, Y. Chai, H. H. Zhang and H. Chen, "Heterogeneous Domain Adaptation With Adversarial Neural Representation Learning: Experiments on E-Commerce and Cybersecurity," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 2, pp. 1862-1875, 1 Feb. 2023, doi: 10.1109/TPAMI.2022.3163338.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.

[4] J. Devlin, M. W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv preprint arXiv:1810.04805, 2019.

Appendix A: Presentation

AI-BASED RESUME RANKING AND CANDIDATE SHORTLISTING SYSTEM

MINI PROJECT FINAL PRESENTATION

Team Members
Pranav C Ajith(U2303182)
Pranav P Narayanan(U2303184)
Sidharth Revi(U2303207)
Satyam Sharma(U2303218)
S6 CSE GAMMA

Guide:
Sangeetha Jamal
Assistant Professor
Department of CSE

OVERVIEW

- Introduction
- Problem Definition
- Project Objective
- Purpose and Need
- Literature Survey
- Proposed Method
- System Architecture
- Design Diagrams
- Results and Outputs
- Assumptions
- Work Breakdown
- Requirements
- Gantt Chart
- Risks and Challenges
- Conclusion
- References

INTRODUCTION

Recruitment processes often involve manually reviewing hundreds of resumes to identify suitable candidates for job positions. Traditional hiring methods are time-consuming, prone to human bias and inefficient when handling large numbers of applications.

This project proposes a Resume Ranking System that automatically analyzes resumes and ranks candidates based on how well their skills match job requirements.

The system uses Natural Language Processing techniques to extract skills and compute similarity scores between resumes and job descriptions.

PROBLEM STATEMENT

To create an automated system to analyze and rank candidates based on job requirements for recruiters receiving large number of resumes using RAG to retrieve relevant experience and LLMs for candidate feedback.

PRIMARY GOAL

The primary goal of this project is to develop an AI-powered system that automatically analyzes resumes, matches them with job descriptions, and ranks candidates using LLMs to help recruiters identify the most suitable applicants quickly and efficiently and RAG to extract relevant features of the resume.

KEY OBJECTIVES

- Automate resume screening to reduce the manual effort required to review large numbers of resumes.
- Rank candidates intelligently based on their skills, experience, and job requirements using NLP techniques.
- Improve recruitment efficiency by helping recruiters quickly identify qualified candidates.
- Provide AI-based explanations using LLM and RAG to clarify why candidates match or do not match a job.
- Build an interactive recruitment platform with separate interfaces for recruiters and candidates.

PURPOSE AND NEED

To simplify the recruitment process using Artificial Intelligence and NLP techniques and assist recruiters in making faster, data-driven hiring decisions.

Need

- The growing number of job applications makes manual resume screening impractical.
- Recruiters require faster and more accurate candidate evaluation.
- Organizations need systems that ensure fair and transparent recruitment.
- AI-driven tools can improve hiring efficiency and decision-making quality.

LITERATURE SURVEY

Author & Year	Methodology	Strengths	Limitations
Devlin et al., 2019	Proposed **BERT**, a transformer-based NLP model that understands contextual relationships in text for tasks such as text classification and semantic matching.	Provides deep contextual understanding of text and improves accuracy in NLP tasks.	Requires high computational resources and large datasets for training.
Lewis et al., 2020	Introduced **Retrieval-Augmented Generation (RAG)** which combines document retrieval with language models to generate context-aware responses.	Improves response accuracy by using external knowledge during generation.	Performance depends on the quality of retrieved information.

Khan & Sharma, 2021	Developed a **resume screening system using NLP and machine learning** to analyze resumes and match them with job descriptions.	Automates the resume screening process and reduces manual effort.	May fail to detect skills written in uncommon formats.
Gupta & Verma, 2022	Proposed an **automated resume parsing system** that extracts skills, education, and experience using NLP techniques.	Efficiently converts unstructured resume text into structured information.	Accuracy depends on resume structure and formatting.
Patel & Shah, 2023	Introduced an **AI-based recruitment system** that ranks candidates using machine learning algorithms based on job requirements.	Provides automated candidate ranking and improves recruitment efficiency.	Cannot effectively evaluate soft skills or personality traits.

PROPOSED METHOD

SYSTEM INTERACTION

The system begins with user interaction where both candidates and recruiters access the platform and perform recruitment-related activities.

- **User Authentication**

  Candidates and recruiters register and log in to access the system.
- **Job Creation**
  Recruiters create job openings by specifying job titles, descriptions, and required skills.
- **Resume Submission**
  Candidates browse available jobs and apply by uploading their resumes.

RESUME PROCESSING

After a resume is submitted, the system processes the uploaded document to extract relevant information for evaluation.

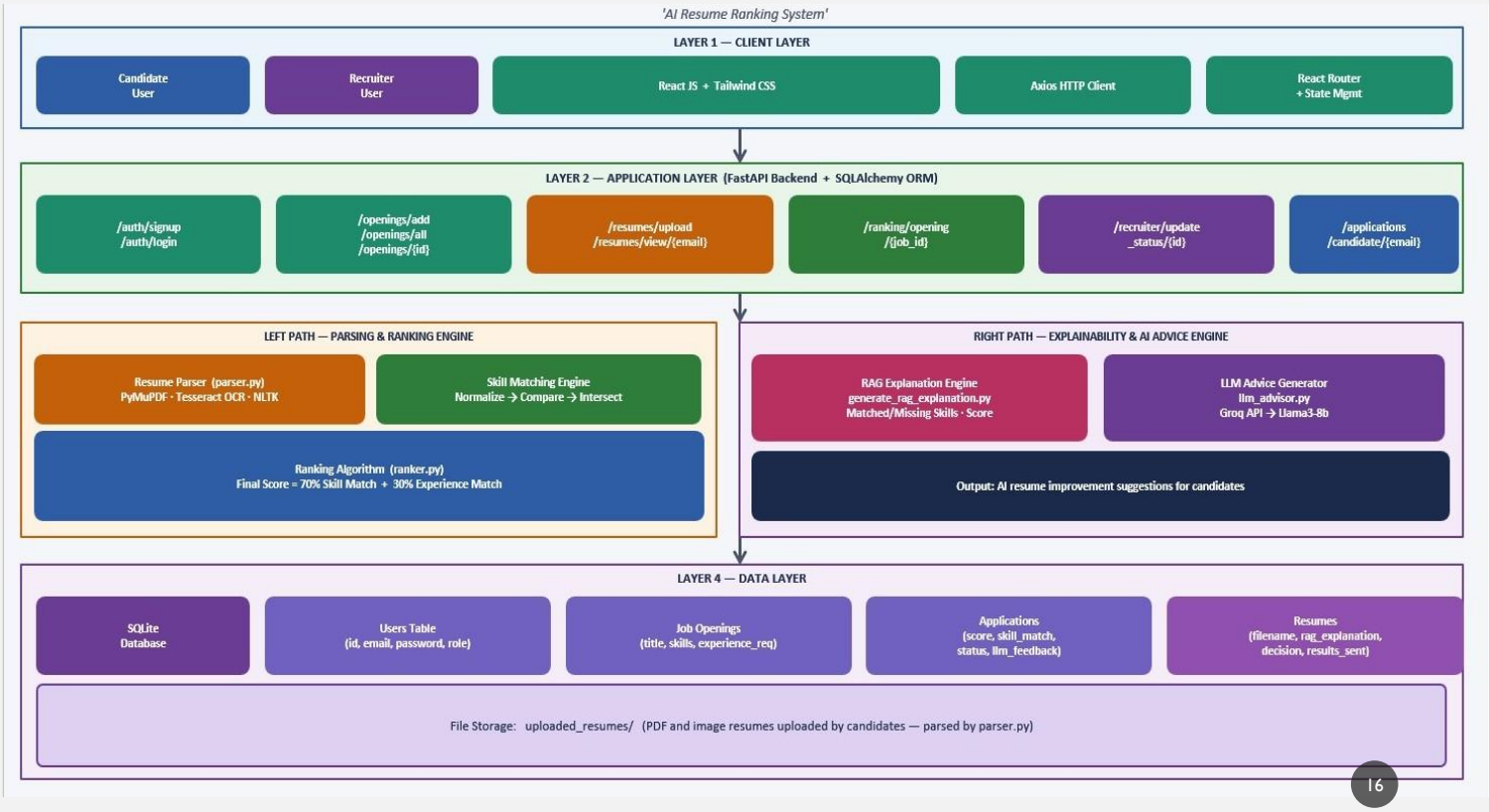
- Resume Parsing
  The system extracts important information such as skills, experience, and keywords from the resume.
- Candidate Ranking
  A ranking algorithm compares the extracted information with job requirements and calculates match scores based on skill and experience alignment.

AI EVALUATION

The system uses artificial intelligence to provide insights and support recruiter decision-making.

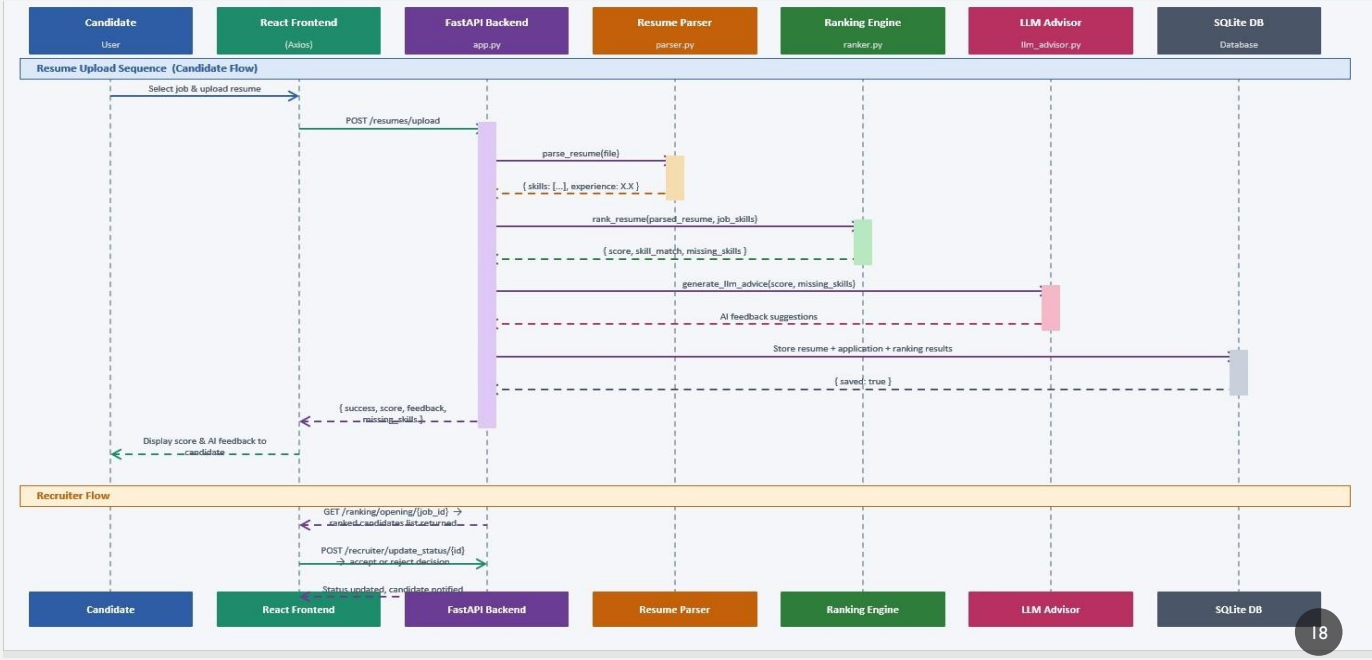
- LLM Explanation Generation
  A Large Language Model generates explanations describing why a candidate scored high or low, highlighting matched and missing skills.
- Recruiter Decision
  Recruiters review the ranked candidates and AI explanations to accept or reject applications.

ARCHITECTURE DIAGRAM

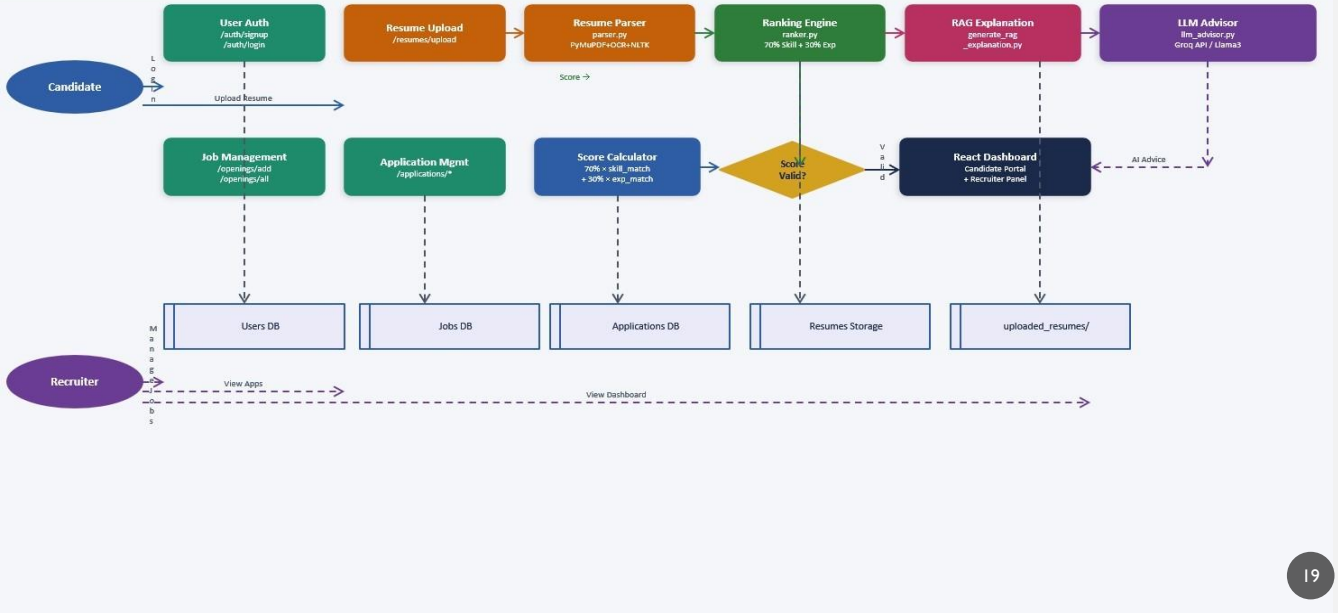


DESIGN DIAGRAMS

SEQUENCE DIAGRAM

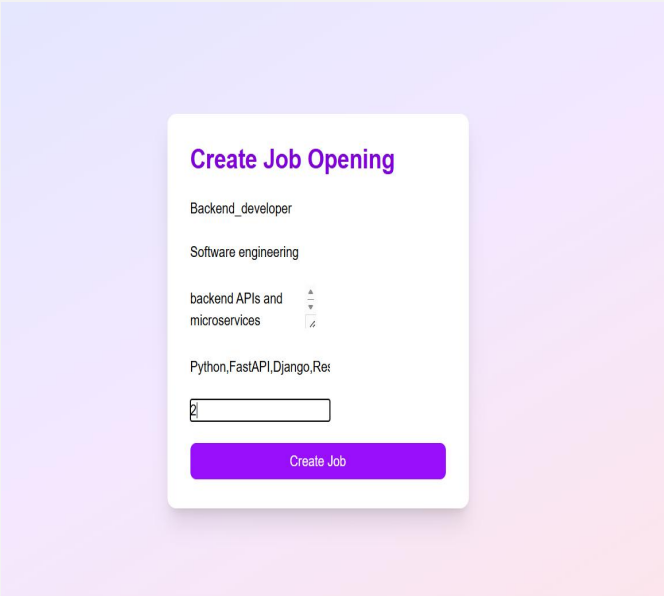


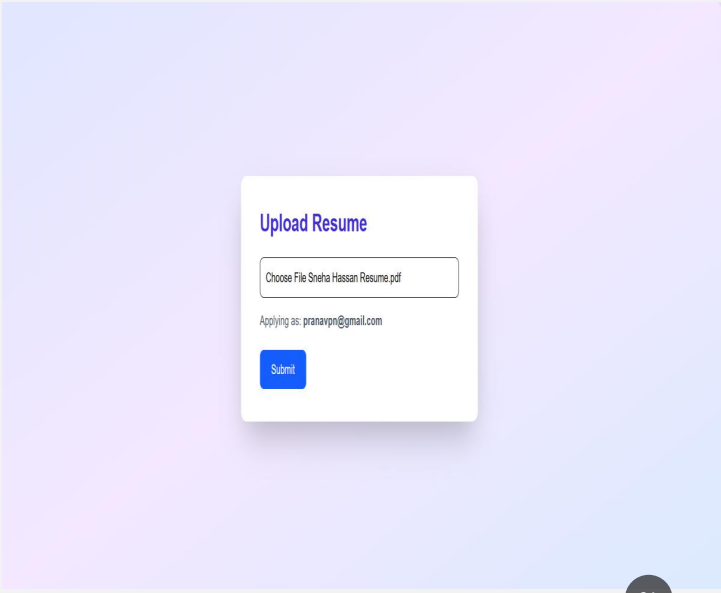
DATAFLOW DIAGRAM



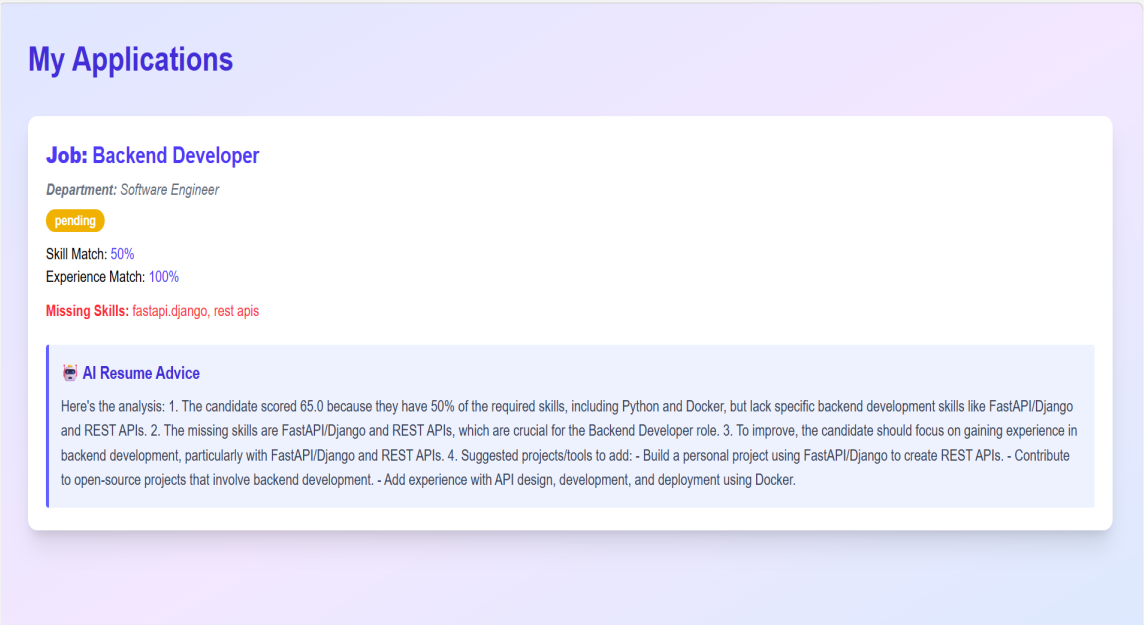
RESULTS

RESUME UPLOAD AND JOB OPENING





RESUME EVALUATION IN PENDING STAGE



RANKING

Rank #1
Candidate: pranavpn@gmail.com
Score: 65
Skill Match: 50 %
Experience Match: 100 %

Missing Skills:
fastapi.django, rest apis

Explain Match | View Resume | Accept | Reject

🤖 AI Explanation

```
JOB ROLE: Backend Developer

MATCHED SKILLS:
python, docker

MISSING SKILLS:
fastapi.django, rest apis

CANDIDATE EXPERIENCE:
2.2 years

REQUIRED EXPERIENCE:
2 years
```


RESULT AND LLM FEEDBACK

My Applications

Job: Backend Developer

Department: Software Engineer

rejected

Skill Match: 25%
Experience Match: 100%

Missing Skills: fastapi.django, python, rest apis

Rank Score: 47.5

🤖 AI Resume Advice

Analysis 1. The candidate scored 47.5 due to their 10.4 years of experience and partial skill match, particularly with "docker". 2. The missing skills are "python", "fastapi.django", and "rest apis", which are crucial for the Backend Developer role. 3. To improve, the candidate should highlight backend development experience and skills, rather than frontend (e.g., ReactJS) and diverse programming languages. 4. Suggested additions: - Personal projects using Python and FastAPI/Django - Experience with REST APIs and microservices - Familiarity with containerization using Docker - Backend-focused certifications or courses to fill skill gaps.

ASSUMPTIONS

- Resumes are uploaded in supported formats such as for processing.
- Job descriptions provided by recruiters clearly specify the required skills and qualifications.
- Candidates provide accurate and truthful information in their resumes.
- The system assumes stable internet connectivity for uploading resumes and accessing the platform.

REQUIREMENTS

Hardware Requirements

- Computer or laptop with minimum 8 GB RAM
- Internet connection for accessing the web application
- Storage space for storing resumes and system data

Software Requirements

- Python (Backend development)
- FastAPI (Backend API framework)
- ReactJS (Frontend development)
- Tailwind CSS (Frontend styling)
- SQLAlchemy (ORM for database interaction)
- SQLite / PostgreSQL (Database)
- Groq API (LLM inference for AI explanation generation)
- NLP and Machine Learning libraries (for resume analysis)

WORK BREAKDOWN

Phase 1 – Problem Understanding & Project Planning (December)

All Members

- Research on challenges in manual resume screening and recruitment inefficiencies
- Study of Artificial Intelligence approaches for resume parsing and candidate-job matching
- Understanding Natural Language Processing techniques for extracting resume information
- Identification of key parameters such as skills, experience, and job requirements for matching
- System architecture planning for an AI-powered resume ranking system

Phase 2 – Dataset Preparation & Resume Processing (December – January)

- Pranav P - Development of resume upload and storage system Implementation of resume text extraction from PDF files Creation of candidate profile structure for storing extracted resume information
- Sidharth - Resume preprocessing and cleaning Identification and extraction of key skills from resumes Handling different resume formats and ensuring consistent parsing
- Pranav C - Creation of job openings database schema Development of job posting functionality for recruiters Structuring job requirements including skills and experience
- Satyam - Data validation and organization of extracted resume features Analysis of extracted resume data for matching readiness Preparation of structured input data for the matching algorithm

Phase 3 – AI Matching Algorithm Development (January – February)

- Pranav P - Implementation of resume parsing logic using NLP techniques Development of skill extraction and keyword matching system Integration of parsed resume data into the backend system
- Sidharth - Development of resume-job matching algorithm Implementation of scoring logic based on skill similarity and experience Testing and refinement of candidate ranking logic
- Pranav C - Implementation of Retrieval Augmented Generation (RAG) explanation system Generation of match explanations such as matched skills and missing skills Integration of AI explanation module with backend APIs
- Satyam - Analysis and optimization of matching scores Testing of algorithm performance across different job profiles Evaluation of ranking accuracy and system reliability

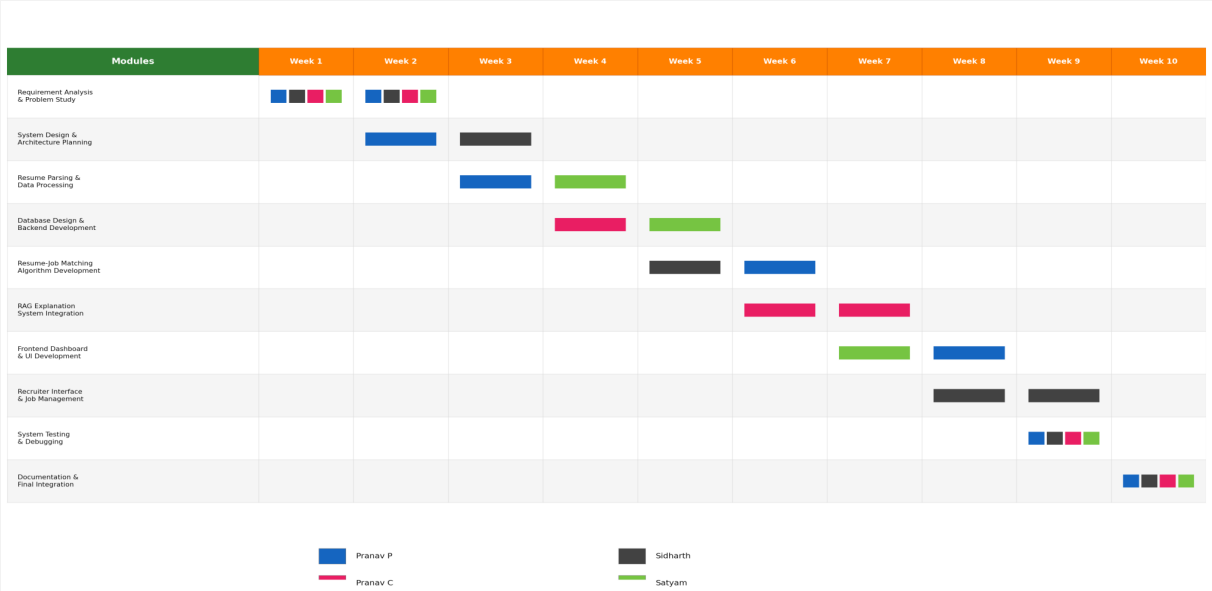
Phase 4 – System Development & Integration (February – March)

- Pranav P - Development of candidate dashboard interface* Implementation of resume upload and job browsing functionality* Integration of frontend with backend APIs
- Sidharth - Development of recruiter dashboard and job posting interface* Implementation of applicant viewing and candidate ranking display* Integration of job management system with database
- Pranav C - Backend development using FastAPI Implementation of APIs for resume processing, job matching, and candidate ranking Database integration using SQLite and ORM models
- Satyam - UI design improvements and dashboard layout optimization* Development of application tracking interface for candidates* Integration of visualization components for match explanations

Phase 5 – Testing, Documentation & Final Submission (March)

- All Members - System testing and debugging of the AI Resume Ranking system Validation of resume parsing, job matching, and ranking functionality Preparation of project presentation and demonstration Documentation of system architecture, algorithms, and implementation Final system integration and submission

GANTT CHART



RISKS AND CHALLENGES

- **Variation in Resume Formats**
  Resumes may be submitted in different formats and structures, making parsing and information extraction challenging.
- **Incomplete Skill Extraction**
  The system may miss certain skills or keywords if they are written in uncommon formats or phrasing.
- **Dependence on Input Quality**
  AI-generated explanations depend on the quality and completeness of the resume and job description data.
- **Technology Integration Complexity**
  Integrating multiple technologies such as frontend frameworks, backend APIs, AI models, and databases can increase system complexity.

CONCLUSION

•The proposed system automates the resume screening process, reducing the time and effort required by recruiters.
•The AI-based ranking mechanism helps identify the most suitable candidates quickly based on skills and experience.
•LLM-generated explanations improve transparency by providing clear reasoning behind candidate rankings.
•Overall, the platform demonstrates how Artificial Intelligence can enhance recruitment efficiency and decision-making.

REFERENCES

- Devlin, J., et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv preprint arXiv:1810.04805.
- Zhang, Y., Chen, X., & Wang, J. (2020). *Automatic Resume Classification and Job Matching Using Natural Language Processing.* IEEE International Conference on Big Data.
- Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems.
- Singh, A., et al. (2023). "Automated Resume Screening using Natural Language Processing." International Journal of Applied Engineering Research.

THANK YOU!

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
RAJAGIRI SCHOOL OF ENGINEERING & TECHNOLOGY (AUTONOMOUS)
RAJAGIRI VALLEY, KAKKANAD, KOCHI, 682039
(Affiliated to APJ Abdul Kalam Technological University)

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and Team work: Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings.

10. Communication: Communicate effectively with the engineering community and with society at large. Be able to comprehend and write effective reports documentation. Make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team. Manage projects in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes

CO1: Identify technically and economically feasible problems.

CO2: Identify and survey the relevant literature for getting exposed to related solutions and software development processes.

CO3: Perform requirement analysis, identify design methodologies and develop adaptable reusable solutions of minimal complexity by using modern tools advanced programming techniques.

CO4: Prepare technical reports and deliver presentations.

CO5: Apply engineering and management principles to achieve the goal of the project.

Appendix C: CO-PO-PSO Mapping

COURSE OUTCOMES (CO)

SNO	Description	Bloom's Taxonomy Level
1	Model and solve real world problems by applying knowledge across domains.	Apply
2	Develop products, processes or technologies for sustainable and socially relevant applications.	Apply
3	Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks.	Apply
4	Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.	Apply
5	Identify technology/research gaps and propose innovative/creative solutions.	Analyze
6	Organize and communicate technical and scientific findings effectively in written and oral forms.	Apply

MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) – PROGRAM SPECIFIC OUTCOMES (PSOs)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	2	2	2	1		2	1	1	1	1	1	2	2	2
CO2	2	2	2		2	3	2	1	1	1	1	2	2	
CO3								3	2	2	1			3
CO4		1	2		3	2	3	2	2	3	1		2	2
CO5	2	3	3	1	2					1	3	3		
CO6			2			2	2	2	3	1	1			3

JUSTIFICATIONS FOR CO-PO MAPPING

Mapping	Level	Justification
101003/ CS822U.CO1- PO1	Medium	Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.
101003/ CS822U.CO1- PO2	Medium	Requires identification, formulation and analysis of engineering problems before developing solutions.
101003/ CS822U.CO1- PO3	Medium	Involves designing systems or solutions to address identified engineering needs.
101003/ CS822U.CO1- PO4	Low	Limited investigation or experimentation may be required to validate the developed models or solutions.
101003/ CS822U.CO1- PO6	Medium	Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications.
101003/ CS822U.CO1- PO7	Medium	Encourages consideration of ethical and professional responsibilities while developing solutions.
101003/ CS822U.CO1- PO8	Low	Some level of teamwork may be required while working on modeling or project tasks.
101003/ CS822U.CO1- PO9	Low	Requires communication of results and findings through reports or presentations.
101003/ CS822U.CO1- PO10	Low	Involves basic planning and execution of tasks while implementing solutions.
101003/ CS822U.CO1- PO11	Low	Encourages independent learning and adapting new knowledge to solve real-world problems.
101003/ CS822U.CO1- PSO1	Medium	Uses core computer science principles to analyze problems and design appropriate technical solutions.
101003/ CS822U.CO1- PSO2	Medium	Involves algorithm design and application of software development practices to implement solutions.
101003/ CS822U.CO1- PSO3	Medium	Applies computer science fundamentals in research or innovative problem solving for societal needs.
101003/ CS822U.CO2- PO1	Medium	Utilizes engineering knowledge to design and develop products or systems.

Mapping	Level	Justification
101003/ CS822U.CO2- PO2	Medium	Requires analysis of user needs and technological feasibility before development.
101003/ CS822U.CO2- PO3	Medium	Focuses on design and development of solutions addressing real-world requirements.
101003/ CS822U.CO2- PO5	Medium	Requires use of modern engineering and IT tools for system or product development.
101003/ CS822U.CO2- PO6	High	Emphasizes sustainability and societal impact during solution development.
101003/ CS822U.CO2- PO7	Medium	Ethical and professional responsibilities are considered when developing socially relevant technologies.
101003/ CS822U.CO2- PO8	Low	Development activities may require collaboration among team members.
101003/ CS822U.CO2- PO9	Low	Involves communicating design ideas and development outcomes.
101003/ CS822U.CO2- PO10	Low	Requires basic planning and coordination during development tasks.
101003/ CS822U.CO2- PO11	Low	Encourages continuous learning of emerging technologies while building solutions.
101003/ CS822U.CO2- PSO1	Medium	Applies computer science fundamentals to design technically feasible solutions.
101003/ CS822U.CO2- PSO2	Medium	Uses programming and software development practices to implement solutions.
101003/ CS822U.CO3- PO8	High	Strongly relates to functioning effectively as a member or leader in multidisciplinary teams.
101003/ CS822U.CO3- PO9	Medium	Requires effective communication among team members during collaboration.
101003/ CS822U.CO3- PO10	Medium	Team leadership involves project coordination and task management.
101003/ CS822U.CO3- PO11	Low	Team interaction supports learning from peers and adapting new approaches.
101003/ CS822U.CO3- PSO3	High	Professional collaboration and teamwork support innovative computer science solutions.

Mapping	Level	Justification
101003/ CS822U.CO4- PO2	Low	Requires analysis and planning before executing project tasks.
101003/ CS822U.CO4- PO3	Medium	Involves designing and implementing project solutions based on planned activities.
101003/ CS822U.CO4- PO5	High	Requires effective use of modern tools and resources during project execution.
101003/ CS822U.CO4- PO6	Medium	Considers societal and environmental impact while implementing solutions.
101003/ CS822U.CO4- PO7	High	Emphasizes adherence to ethical practices and professional responsibilities.
101003/ CS822U.CO4- PO8	Medium	Task execution may involve teamwork and coordination among team members.
101003/ CS822U.CO4- PO9	Medium	Requires communication of progress, outcomes and documentation.
101003/ CS822U.CO4- PO10	High	Focuses on project management including scheduling, resource allocation and task execution.
101003/ CS822U.CO4- PO11	Low	Encourages adaptability and continuous learning during project implementation.
101003/ CS822U.CO4- PSO2	Medium	Uses software development practices to execute and manage project tasks.
101003/ CS822U.CO4- PSO3	Medium	Applies computer science knowledge in practical project development environments.
101003/ CS822U.CO5- PO1	Medium	Uses engineering knowledge to understand technological limitations and opportunities.
101003/ CS822U.CO5- PO2	High	Requires deep problem analysis and identification of research gaps.
101003/ CS822U.CO5- PO3	High	Proposes innovative design solutions to address identified gaps.
101003/ CS822U.CO5- PO4	Low	May involve limited investigation or validation of proposed ideas.
101003/ CS822U.CO5- PO5	Medium	Uses appropriate tools and techniques to analyze and develop innovative solutions.
101003/ CS822U.CO5- PO10	Low	Requires minimal planning while proposing new solution approaches.

Mapping	Level	Justification
101003/ CS822U.CO5- PO11	High	Strongly promotes independent learning and exploration of emerging technologies.
101003/ CS822U.CO5- PSO1	High	Applies computer science fundamentals to identify and solve complex technical problems.
101003/ CS822U.CO6- PO3	Medium	Requires presentation of design concepts and developed solutions.
101003/ CS822U.CO6- PO6	Medium	Communication may include discussion of societal or environmental implications.
101003/ CS822U.CO6- PO7	Medium	Requires responsible and ethical communication of technical results.
101003/ CS822U.CO6- PO8	Medium	Communication supports collaboration within team environments.
101003/ CS822U.CO6- PO9	High	Strongly relates to effective written and oral communication of technical findings.
101003/ CS822U.CO6- PO10	Low	Requires structured reporting and documentation in project management contexts.
101003/ CS822U.CO6- PO11	Low	Encourages continuous improvement in communication and presentation skills.
101003/ CS822U.CO6- PSO3	High	Communication of innovative computer science solutions and research outcomes.